Լաբորատոր աշխատանք 11

1. Աշխատանքի նպատակը

Գործնականում հասկանալ և տարբերակել C++ լեզվում օբյեկտի փոխանցման 3 հիմնական եղանակները՝ **արժեքով (by value)**, **հղումով (by reference)** և **հաստատուն հղումով (by const reference)**, ինչպես նաև սովորել ապահովել տվյալների փակ պահպանումը (encapsulation) դասի ներսում:

2. Աշխատանքի պահանջները

- **Դասիառուցվածք:**
    - Ստեղծել SubscriptionUsage դաս, որտեղ տվյալները պահվում են private std::vector<int> (ծառայության միավորներ) և int p (ամսական անվճար սահմանաչափ) դաշտերում:
    - Իրականացնել պարամետրերով կոնստրուկտոր, print() և set(index, value) մեթոդներ, ինչպես նաև public ընթերցման մեթոդներ:
- **Վավերացում և Անվտանգություն:**
    - Հավաքածուի չափը պետք է լինի $3 \leq n \leq 30$:
    - Օգտագործված միավորները և p-ն պետք է լինեն ոչ բացասական ամբողջ թվեր:
    - Սխալ տվյալների կամ ինդեքսների դեպքում մեթոդները պետք է վերադարձնեն false կամ կանխեն օբյեկտի սխալ ստեղծումը:
- **Պահանջվող 3 արտաքին ֆունկցիաները (առանց friend բանալի բառի):**
    1. analyze(const SubscriptionUsage&) (Const հղում) — Արտածում է 3 հաշվարկ.
        - Գերազանցման ընդհանուր վճարը ($\sum \max(0, \text{usage} - p) \cdot 50$ դրամ):
        - Ամենափոքր օգտագործմամբ հաճախորդի ինդեքսը (հավասարության դեպքում՝ առաջինը):
        - Անվճար սահմանաչափը չգերազանցած ($usage \leq p$) հաճախորդների քանակը:
    2. changeCopy(SubscriptionUsage object, int index, double value) (Արժեքով) — Փոփոխում և արտածում է **միայն տեղային պատճենը**:
    3. reversed(const SubscriptionUsage& object) (Const հղում) — Վերադարձնում է **նոր օբյեկտ**՝ շրջված զանգվածով և նույն p-ով:

3. Տեսական մաս

- **Արժեքով փոխանցում (by value):**
    - Ֆունկցիային փոխանցելիս ստեղծվում է օբյեկտի **ամբողջական պատճենը** (կանչվում է պատճենող կոնստրուկտորը):
    - Ֆունկցիայի ներսում կատարված բոլոր փոփոխությունները կատարվում են միայն տեղային պատճենի վրա և **չեն ազդում բնօրինակի վրա**:
- **Հղումով փոխանցում (by reference - T&):**
    - Ֆունկցիային է փոխանցվում բնօրինակ օբյեկտի հասցեն (ալիասը):

- Պատճենում չի կատարվում (արագագործ է), և ֆունկցիայի ներսում արված փոփոխությունները **ուղղակիորեն փոխում են սկզբնական օբյեկտը**:

- **Հաստատուն հղումով փոխանցում (by const reference - const T&):**
  - Օբյեկտը փոխանցվում է հղումով, այսինքն՝ պատճենում տեղի չի ունենում (հիշողության և ժամանակի խնայողություն):
  - const բանալի բառը **երաշխավորում է, որ ֆունկցիան չի կարող փոփոխել** օբյեկտի տվյալները (թույլատրվում է կանչել միայն const նշված մեթոդները):

- **Տվյալների պաշտպանություն (Encapsulation):**
  - Դասի մասնավոր (private) տվյալներին արտաքին սովորական ֆունկցիաները հասնում են բացառապես հանրային (public) ընթերցող մեթոդների (getters) միջոցով:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <stdexcept>

class SubscriptionUsage {
private:
    std::vector<int> data;
    int p;

public:
    // Պարամետրերով կոնստրուկտոր՝ վավերացմամբ
    SubscriptionUsage(const std::vector<int>& usageList, int freeLimit) {
        if (usageList.size() < 3 || usageList.size() > 30 || freeLimit < 0) {
            throw std::invalid_argument("Սխալ չափ (պետք է լինի 3-ից 30) կամ բացասական սահմանաչափ p:");
        }
        for (int val : usageList) {
            if (val < 0) {
                throw std::invalid_argument("Օգտագործման արժեքները չեն կարող լինել բացասական:");
            }
        }
        data = usageList;
```

```
    p = freeLimit;
}

// Public ընթերցման մեթոդներ
int size() const {
    return data.size();
}

int getP() const {
    return p;
}

int get(int index) const {
    if (index < 0 || index >= (int)data.size()) {
        throw std::out_of_range("Ինդեքսը դուրս է սահմաններից:");
    }
    return data[index];
}

// Փոփոխող մեթոդ՝ ստուգումներով
bool set(int index, int value) {
    if (index < 0 || index >= (int)data.size() || value < 0) {
        return false;
    }
    data[index] = value;
    return true;
}

// Արտածման մեթոդ
void print() const {
    std::cout << "p = " << p << ", Ծախսեր (usage) = [";
    for (size_t i = 0; i < data.size(); ++i) {
```

```
        std::cout << data[i] << (i + 1 < data.size() ? ", " : "");
    }
    std::cout << "]" << std::endl;
}

// Հաշվարկ 1: Գերազանցման ընդհանուր վճարը
long long calculateTotalExcessFee() const {
    long long totalFee = 0;
    for (int usage : data) {
        if (usage > p) {
            totalFee += static_cast<long long>(usage - p) * 50;
        }
    }
    return totalFee;
}

// Հաշվարկ 2: Ամենափոքր օգտագործմամբ հաճախորդի ինդեքսը (հավասարների դեպքում՝ առաջինը)
int findMinUsageIndex() const {
    if (data.empty()) return -1;
    int minVal = data[0];
    int minIdx = 0;
    for (size_t i = 1; i < data.size(); ++i) {
        if (data[i] < minVal) {
            minVal = data[i];
            minIdx = i;
        }
    }
    return minIdx;
}

// Հաշվարկ 3: Անվճար սահմանաչափը չգերազանցած հաճախորդների քանակը (usage <= p)
int countWithinLimit() const {
```

```
        int count = 0;
        for (int usage : data) {
            if (usage <= p) {
                count++;
            }
        }
        return count;
    }

    // Զանգվածի շրջման մեթոդ
    void reverse() {
        std::reverse(data.begin(), data.end());
    }
};

// Արտաքին ֆունկցիաներ (առանց friend-ի)

// 1. analyze - ընթերցում է const հղումով և արտածում հաշվարկները
void analyze(const SubscriptionUsage& object) {
    std::cout << "\n--- analyze (const հղում) ---" << std::endl;
    std::cout << "Գերազանցման ընդհանուր վճար: " << object.calculateTotalExcessFee() << " դրամ" << std::endl;
    std::cout << "Ամենափոքր օգտագործման ինդեքսը: " << object.findMinUsageIndex()
              << " (Արժեքը: " << object.get(object.findMinUsageIndex()) << ")" << std::endl;
    std::cout << "Սահմանաչափը չգերազանցածների քանակը: " << object.countWithinLimit() << std::endl;
    std::cout << "-----------------------------" << std::endl;
}

// 2. changeCopy - փոփոխում և արտածում է միայն տեղային պատճենը (արժեքով փոխանցում)
void changeCopy(SubscriptionUsage object, int index, int value) {
    std::cout << "\n--- changeCopy (արժեքով փոխանցում - պատճեն) ---" << std::endl;
    if (object.set(index, value)) {
        std::cout << "Տեղային պատճենը հաջողությամբ փոփոխվեց:" << std::endl;
```

```
    } else {
        std::cout << "Փոփոխությունը ձախողվեց (սխալ ինդեքս կամ արժեք):" << std::endl;
    }
    object.print();
    std::cout << "-----------------------------------------------" << std::endl;
}

// 3. reversed - վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով
SubscriptionUsage reversed(const SubscriptionUsage& object) {
    SubscriptionUsage result = object;
    result.reverse();
    return result;
}

int main() {
    try {
        // Փորձնական տվյալներ №1 (առաջադրանքից)
        std::cout << "=== ՓՈՐՁԱՐԿՈՒՄ №1 ===" << std::endl;
        SubscriptionUsage sub1({20, 55, 40, 80}, 50);

        std::cout << "Բնօրինակ օբյեկտը ստեղծելուց հետո:" << std::endl;
        sub1.print();

        analyze(sub1);
        sub1.print(); // Ստուգում, որ բնօրինակը չի փոխվել

        changeCopy(sub1, 0, 90);
        sub1.print(); // Ստուգում, որ բնօրինակը չի փոխվել

        SubscriptionUsage sub1_rev = reversed(sub1);
        std::cout << "Շրջված նոր օբյեկտը:" << std::endl;
        sub1_rev.print();
```

```
        std::cout << "Բնօրինակը շրջվածից հետո (անփոփոխ):" << std::endl;
        sub1.print();

        // Փորձարկման տվյալներ №2
        std::cout << "\n=== ՓՈՐՁԱՐԿՈՒՄ №2 ===" << std::endl;
        SubscriptionUsage sub2({10, 30, 30, 45, 50}, 40);
        sub2.print();
        analyze(sub2);
        changeCopy(sub2, 2, 60);

        // Փորձարկման տվյալներ №3
        std::cout << "\n=== ՓՈՐՁԱՐԿՈՒՄ №3 ===" << std::endl;
        SubscriptionUsage sub3({60, 70, 80}, 50);
        sub3.print();
        analyze(sub3);
        changeCopy(sub3, 1, 20);

    } catch (const std::exception& e) {
        std::cerr << "Սխալ: " << e.what() << std::endl;
    }

    return 0;
}
```